WTC_ AI Curriculum

Exercise: Performance Optimization Challenge

Exercise Instructions

1. Select one of the following performance scenarios (each designed for a different performance issue):
   - Slow Code Analysis: Python data processing function
   - Memory Usage Optimization: Java image processing application
   - Slow Database Query Analysis: JavaScript/Node.js application with database access
2. Use the corresponding AI prompt from the "Identifying Performance Bottlenecks" section to analyze the performance issue
3. Implement the suggested optimizations
4. Measure performance before and after your changes
5. Document your optimization process, results, and key learnings

IMPORTANT

Check out the starter code at this Gitlab Repo. For reference the specific code is also listed on this page.

Sample Performance Issues for Analysis

1. Slow Code Analysis (Python)

Code with Performance Issue:

```python
# inventory_analysis.py
def find_product_combinations(products, target_price, price_margin=10):
    """
    Find all pairs of products where the combined price is within
    the target_price ± price_margin range.

    Args:
        products: List of dictionaries with 'id', 'name', and 'price' keys
        target_price: The ideal combined price
        price_margin: Acceptable deviation from the target price

    Returns:
        List of dictionaries with product pairs and their combined price
```

```
    """
    results = []

    # For each possible pair of products
    for i in range(len(products)):
        for j in range(len(products)):
            # Skip comparing a product with itself
            if i != j:
                product1 = products[i]
                product2 = products[j]

                # Calculate combined price
                combined_price = product1['price'] + product2['price']

                # Check if the combined price is within the target range
                if (target_price - price_margin) <= combined_price <= (target_price
+ price_margin):
                    # Avoid duplicates like (product1, product2) and (product2,
product1)
                    if not any(r['product1']['id'] == product2['id'] and
                               r['product2']['id'] == product1['id'] for r in
results):

                        pair = {
                            'product1': product1,
                            'product2': product2,
                            'combined_price': combined_price,
                            'price_difference': abs(target_price - combined_price)
                        }
                        results.append(pair)

    # Sort by price difference from target
    results.sort(key=lambda x: x['price_difference'])
    return results

# Example usage
if __name__ == "__main__":
    import time
    import random

    # Generate a large list of products
    product_list = []
    for i in range(5000):
        product_list.append({
            'id': i,
            'name': f'Product {i}',
            'price': random.randint(5, 500)
        })
```

```
    # Measure execution time
    start_time = time.time()
    combinations = find_product_combinations(product_list, 500, 50)
    end_time = time.time()

    print(f"Found {len(combinations)} product combinations")
    print(f"Execution time: {end_time - start_time:.2f} seconds")
```

Context for the Prompt:

- This code is used in an e-commerce application to suggest product pairs that match a target price point
- The function typically processes 5,000+ products
- Current execution time: Approximately 20-30 seconds for 5,000 products
- The function runs whenever a user visits the "Product Recommendations" page
- Environment: Python 3.9 on a web server with 4GB RAM

2. Memory Usage Optimization (Java)

Code with Performance Issue:

```
// ImageProcessor.java
import javax.imageio.ImageIO;
import java.awt.image.BufferedImage;
import java.io.File;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

public class ImageProcessor {

    public static void main(String[] args) {
        // Simulate processing a batch of images
        try {
            processImageFolder("sample_images", "processed_images");
        } catch (OutOfMemoryError e) {
            System.err.println("Out of memory error occurred: " + e.getMessage());
            e.printStackTrace();
        } catch (IOException e) {
            System.err.println("IO error: " + e.getMessage());
            e.printStackTrace();
        }
    }
```

```
    public static void processImageFolder(String inputFolder, String outputFolder)
throws IOException {
        File folder = new File(inputFolder);
        File[] imageFiles = folder.listFiles((dir, name) ->
                name.toLowerCase().endsWith(".jpg") ||
                name.toLowerCase().endsWith(".png"));

        if (imageFiles == null || imageFiles.length == 0) {
            System.out.println("No images found in the folder");
            return;
        }

        // Create output directory if it doesn't exist
        File outputDir = new File(outputFolder);
        if (!outputDir.exists()) {
            outputDir.mkdirs();
        }

        // Store all images in memory first
        List<BufferedImage> images = new ArrayList<>();
        System.out.println("Loading all images into memory...");

        for (File imageFile : imageFiles) {
            BufferedImage image = ImageIO.read(imageFile);
            images.add(image);
            System.out.println("Loaded: " + imageFile.getName());
        }

        // Process all images
        System.out.println("Processing all images...");
        List<BufferedImage> processedImages = new ArrayList<>();

        for (BufferedImage image : images) {
            BufferedImage processed = applyEffects(image);
            processedImages.add(processed);
        }

        // Save all processed images
        System.out.println("Saving all processed images...");
        for (int i = 0; i < imageFiles.length; i++) {
            String outputName = outputFolder + File.separator + "processed_" +
imageFiles[i].getName();
            ImageIO.write(processedImages.get(i),
getImageFormat(imageFiles[i].getName()), new File(outputName));
            System.out.println("Saved: " + outputName);
        }
```

```
        System.out.println("All images processed successfully");
    }

    private static BufferedImage applyEffects(BufferedImage original) {
        // Simulate memory-intensive image processing
        int width = original.getWidth();
        int height = original.getHeight();

        // Create a new image with the same dimensions
        BufferedImage processed = new BufferedImage(width, height,
original.getType());

        // Process each pixel - simple grayscale conversion for this example
        for (int y = 0; y < height; y++) {
            for (int x = 0; x < width; x++) {
                int rgb = original.getRGB(x, y);

                int alpha = (rgb >> 24) & 0xff;
                int red = (rgb >> 16) & 0xff;
                int green = (rgb >> 8) & 0xff;
                int blue = rgb & 0xff;

                // Convert to grayscale
                int gray = (red + green + blue) / 3;

                // Set new RGB
                int newRGB = (alpha << 24) | (gray << 16) | (gray << 8) | gray;
                processed.setRGB(x, y, newRGB);
            }
        }

        return processed;
    }

    private static String getImageFormat(String filename) {
        if (filename.toLowerCase().endsWith(".png")) {
            return "png";
        } else {
            return "jpeg";
        }
    }
}
```

Error Message:

```
Exception in thread "main" java.lang.OutOfMemoryError: Java heap space
        at java.base/java.awt.image.DataBufferInt.<init>(DataBufferInt.java:75)
        at java.base/java.awt.image.Raster.createPackedRaster(Raster.java:467)
        at
java.base/java.awt.image.DirectColorModel.createCompatibleWritableRaster(DirectColo
rModel.java:1032)
        at java.base/java.awt.image.BufferedImage.<init>(BufferedImage.java:350)
        at ImageProcessor.applyEffects(ImageProcessor.java:64)
        at ImageProcessor.processImageFolder(ImageProcessor.java:47)
        at ImageProcessor.main(ImageProcessor.java:16)
```

Context for the Prompt:

- This Java application processes a batch of images (applying a grayscale effect)
- The error occurs when processing about 50-100 high-resolution images (2000x1500 pixels each)
- The application is running with default JVM memory settings (256MB)
- The application crashes with OutOfMemoryError on larger batches of images
- Environment: Java 11, Windows 10 with 8GB RAM

3. Slow Database Query Analysis (JavaScript/Node.js)

Code with Performance Issue:

```javascript
// orders-service.js
const { Pool } = require('pg');

// Database connection
const pool = new Pool({
  user: 'app_user',
  host: 'localhost',
  database: 'ecommerce',
  password: 'password123',
  port: 5432,
});

async function getCustomerOrderDetails(customerId, startDate, endDate) {
  console.time('orderQueryTime');

  try {
    // Query to get all order details for a customer
    const result = await pool.query(`
      SELECT
        o.order_id,
        o.order_date,
```

```
        o.total_amount,
        o.status,
        c.customer_name,
        c.email,
        (
          SELECT json_agg(
            json_build_object(
              'product_id', p.product_id,
              'product_name', p.name,
              'quantity', oi.quantity,
              'unit_price', p.price,
              'subtotal', (oi.quantity * p.price)
            )
          )
          FROM order_items oi
          JOIN products p ON oi.product_id = p.product_id
          WHERE oi.order_id = o.order_id
        ) as items,
        (
          SELECT json_agg(
            json_build_object(
              'status', s.status,
              'date', s.status_date,
              'notes', s.notes
            )
          )
          FROM order_status_history s
          WHERE s.order_id = o.order_id
          ORDER BY s.status_date DESC
        ) as status_history,
        a.street,
        a.city,
        a.state,
        a.postal_code,
        a.country
      FROM orders o
      JOIN customers c ON o.customer_id = c.customer_id
      LEFT JOIN addresses a ON o.shipping_address_id = a.address_id
      WHERE o.customer_id = $1
        AND o.order_date BETWEEN $2 AND $3
      ORDER BY o.order_date DESC
    `, [customerId, startDate, endDate]);

    console.timeEnd('orderQueryTime');
    return result.rows;
  } catch (err) {
    console.error('Database query error:', err);
    throw err;
```

```
  }
}

// Example usage in Express route handler
async function getOrdersHandler(req, res) {
  try {
    const { customerId } = req.params;
    const { startDate = '2023-01-01', endDate = '2023-12-31' } = req.query;

    const orders = await getCustomerOrderDetails(customerId, startDate, endDate);

    res.json({
      success: true,
      count: orders.length,
      data: orders
    });
  } catch (error) {
    console.error('Error in getOrdersHandler:', error);
    res.status(500).json({
      success: false,
      error: 'An error occurred while fetching orders'
    });
  }
}

module.exports = {
  getCustomerOrderDetails,
  getOrdersHandler
};
```

Database Context:

- PostgreSQL 13 database with the following tables:
  - orders (~100,000 rows)
  - customers (~10,000 rows)
  - order_items (~500,000 rows)
  - products (~5,000 rows)
  - order_status_history (~300,000 rows)
  - addresses (~15,000 rows)
- Indexes: Primary keys are indexed, but no additional indexes
- The query takes 8-10 seconds to execute for customers with many orders

- The slow response time is causing timeout issues in the web application
- Environment: Node.js 14 running on a server with 4 CPU cores and 8GB RAM

Example Prompt Completion (for reference)

Here's how you might complete the "Slow Code Analysis" prompt for the Python example:

```
I have a piece of code that's running slowly. I'd like to understand why and how to
improve it.

Here's the slow-performing code:

def find_product_combinations(products, target_price, price_margin=10):
    """
    Find all pairs of products where the combined price is within
    the target_price ± price_margin range.

    Args:
        products: List of dictionaries with 'id', 'name', and 'price' keys
        target_price: The ideal combined price
        price_margin: Acceptable deviation from the target price

    Returns:
        List of dictionaries with product pairs and their combined price
    """
    results = []

    # For each possible pair of products
    for i in range(len(products)):
        for j in range(len(products)):
            # Skip comparing a product with itself
            if i != j:
                product1 = products[i]
                product2 = products[j]

                # Calculate combined price
                combined_price = product1['price'] + product2['price']

                # Check if the combined price is within the target range
                if (target_price - price_margin) <= combined_price <= (target_price
+ price_margin):
                    # Avoid duplicates like (product1, product2) and (product2,
product1)
                    if not any(r['product1']['id'] == product2['id'] and
                               r['product2']['id'] == product1['id'] for r in
```

```
results):

                        pair = {
                            'product1': product1,
                            'product2': product2,
                            'combined_price': combined_price,
                            'price_difference': abs(target_price - combined_price)
                        }
                        results.append(pair)

    # Sort by price difference from target
    results.sort(key=lambda x: x['price_difference'])
    return results

Context about the issue:
- What this code is supposed to do: Find pairs of products from our inventory that
add up to roughly a target price (within a margin)
- Typical input size/data: Processing 5,000+ products in our inventory
- Current performance: Takes about 25 seconds to run, making our product
recommendation page slow to load
- Environment: Python 3.9 running on our web server with 4GB RAM

Could you please:
1. Explain in simple terms why this code might be slow
2. Identify the specific operations or patterns that are likely causing the
slowdown
3. Suggest 2-3 specific improvements I could make
4. Explain the performance concepts I should learn to avoid similar issues in the
future
5. If there are any tools or techniques I could use to measure the actual
bottlenecks, please suggest them

I'm particularly interested in learning the underlying performance concepts, not
just getting a quick fix.
```

Reflection Questions

- How did the optimization change your understanding of the algorithm, memory management, or database query patterns?
- What performance improvements did you achieve? Were they significant enough to justify the code changes?
- What did you learn about performance bottlenecks that you didn't know before?
- How would you approach similar performance issues in the future?

- What tools or techniques would you use to identify similar issues proactively?
